

Phase 3.5 Item #3 — Real Hardware Support: Planning Doc

Status: Not started. This is a scoping document for a future dedicated session — item #3 is broader than its original one-line roadmap description ("real standalone hardware drivers") suggested. It touches nearly the entire driver layer, not just storage, and is closer to its own sub-phase than a single feature.

Goal: YouOS boots and runs on a real physical machine, using its own drivers, without relying on QEMU/KVM-specific behavior anywhere in the kernel.

No real hardware available yet for testing — this doc is prep/audit work: identify what's already real-hardware-compatible, what's QEMU-specific and needs replacing, and what's simply missing. Actual bring-up and debugging on physical hardware is a separate future effort once a target machine (or bootable USB test rig) is available.

1. The big gap: no PCI bus enumeration

Skimming `kernel/meson.build`'s source list, there is no `pci.c` or equivalent anywhere in the kernel. This is likely the single biggest blocker for real hardware, bigger than any individual driver:

- USB controllers (UHCI, and any future xHCI), network cards (RTL8139 for item #5), and AHCI storage controllers are all **PCI devices** — their I/O port base or MMIO base address is not fixed; it's assigned by firmware at boot and must be discovered via PCI configuration space enumeration (`0xCF8/0xCFC` I/O ports, or `MMCONFIG` on newer systems).
- If `uhci.c` currently hardcodes a fixed I/O base, that's almost certainly a QEMU coincidence (QEMU's `piix3` USB controller happens to live at a predictable address) — real hardware will assign a different address, and the driver will silently fail to find its device.
- **Action item:** write a PCI enumeration driver (`kernel/drivers/pci.c`) before any of the PCI-attached drivers below can be made real-hardware-safe. This should probably be the *first* concrete task when this item is picked up, since everything else depends on it.

2. Storage

- **ATA/IDE** (`kernel/drivers/ata.c`) — real PATA protocol over fixed legacy I/O ports (`0x1F0/0x3F6` etc.), not PCI-discovered, so this one genuinely doesn't need PCI enumeration to keep working. Should work on real hardware *that has a legacy IDE controller or IDE-compatibility mode enabled in firmware*.
- **Gap:** most machines from roughly the last 10-15 years default to AHCI (and increasingly NVMe) with no IDE-compatibility fallback available at all. An ATA-only driver may simply find no disk on a lot of real hardware.

- **Action item:** decide whether to require IDE-compatibility mode as a documented limitation (simplest), or write an AHCI driver (bigger effort, needs PCI enumeration first, but far more likely to actually find a disk on real hardware).

3. USB

- **UHCI** (`kernel/drivers/uhci.c`) already exists — genuine USB 1.1 controller protocol, not QEMU-specific in itself.
- **Gap:** UHCI is Intel's older, USB 1.1-only controller standard. Virtually all real hardware from the last ~15 years uses xHCI (USB 3, backward-compatible with 1.1/2.0 devices but via a completely different register interface, not UHCI-compatible). A UHCI-only driver likely finds **no USB controller at all** on most modern real machines.
- Needs PCI enumeration (see #1) to locate the controller's base address regardless of which standard is targeted.
- **Action item:** this is probably the largest single piece of new driver work in this whole item — an xHCI driver is significantly more complex than UHCI. Worth scoping as its own sub-effort, possibly starting with "detect xHCI presence and fail gracefully" before attempting full xHCI support.

4. Keyboard / Mouse

- Currently PS/2 (`kernel/drivers/keyboard.c`, `kernel/drivers/mouse.c`) — a real, standard hardware protocol via fixed I/O ports (0x60/0x64), not QEMU-specific, and not PCI-based.
- **Gap:** many real modern laptops/motherboards have no physical PS/2 controller at all; USB keyboards/mice only work via PS/2 emulation ("USB legacy support") in firmware, which isn't universal or guaranteed on. If that firmware option is off, or absent, PS/2 driver finds nothing.
- **Action item:** true robustness eventually needs USB HID support (keyboard/mouse as USB devices) as a fallback, which depends on the USB stack above — but PS/2 alone is a reasonable first real-hardware milestone, since most desktop motherboards still support USB-legacy-as-PS/2.

5. Framebuffer / Display

- Comes via GRUB's multiboot2 framebuffer tag (`kernel/drivers/fb.c`), which GRUB populates from real VESA VBE or UEFI GOP on physical hardware essentially the same way it does under QEMU — this is one of the more likely-to-already-work pieces.
- **Action item (low risk, but verify):** audit `fb.c/kernel_main.c`'s framebuffer setup for any assumption specific to QEMU's reported resolution/pitch/format defaults, rather than reading whatever GRUB actually reports.

6. RAM detection

- Presumably comes from multiboot2's memory map tag, parsed at boot for the PMM.

- **Gap:** QEMU's memory map is unusually simple/contiguous. Real hardware memory maps have holes, reserved regions (ACPI reclaimable/NVS, memory-mapped device regions, etc.) — a parser that assumes a simple contiguous layout could allocate into a reserved region and corrupt firmware state or hang.
- **Action item:** audit the memory-map-parsing code specifically for handling of non-USABLE region types, not just "trust every entry as free RAM."

7. Power management (shutdown/restart)

- **Gap, likely significant:** `do_shutdown()`/`do_restart()` almost certainly currently work via a QEMU-specific mechanism — either the `isa-debug-exit` device (`iobase=0x604`, explicitly a QEMU debug feature, doesn't exist on real hardware at all) or QEMU's ACPI shutdown shortcut port. Neither exists on physical hardware.
- Real hardware shutdown/restart requires either real ACPI (parsing ACPI tables, executing the S5 sleep-state AML method — genuinely complex) or the simpler legacy "triple fault" / keyboard-controller-pulse trick for restart specifically (well-known technique, much simpler than full ACPI).
- **Action item:** restart via keyboard-controller reset pulse is a reasonably contained, well-documented technique — good candidate for an early, self-contained real-hardware fix. Shutdown is harder (needs at least minimal ACPI parsing) — may be worth deferring "real shutdown" and just documenting "use the physical power button" as an acceptable interim limitation.

8. Interrupt controller (PIC vs APIC)

- Currently PIC-only (`kernel/arch/x86_64/pic.c`, per the boot log's "PIC initialized"), which is legacy but broadly still present/emulatable on real hardware via chipset compatibility — likely fine as a baseline.
- **Gap:** real multi-core support (see #9) needs the Local APIC and I/O APIC, not the legacy PIC — this becomes relevant if/when SMP is ever attempted, but isn't blocking for a single-core real-hardware boot.

9. CPU — SMP / multi-core

- Not mentioned anywhere in the codebase seen so far — the scheduler (`kernel/proc/scheduler.c`) appears to assume a single core.
- Real hardware is virtually always multi-core; QEMU with `-cpu host` running single-vCPU masks this entirely.
- **Out of scope for "does it boot on real hardware at all,"** but worth flagging: a single-core-only kernel is a real, deliberate limitation to document, not an oversight — full SMP bring-up (AP startup via APIC IPI, per-core state) is a substantial separate effort beyond "make it boot on real hardware."

10. Boot media / GRUB

- GRUB itself abstracts most of the BIOS-vs-UEFI, VBE-vs-GOP differences already — this is likely the *least* risky part of the whole item, since GRUB's multiboot2 handoff is designed to normalize exactly these differences.
 - **Action item:** confirm `grub-mkrescue`'s output actually boots via both legacy BIOS and UEFI on real firmware (not just QEMU's OVMF/SeaBIOS), and that the target boot media (USB stick) is written correctly (dd-style raw write of the ISO, not a filesystem copy) — this is a testing-logistics item, not a code item.
-

Suggested priority order (once a session is dedicated to this)

1. **PCI enumeration driver** — blocks nearly everything else (USB, any future AHEAD/AHCI or network work).
2. **Power management** — restart via keyboard-controller pulse (self-contained, useful even before anything else here is done).
3. **RAM/memory-map audit** — cheap to check, high blast radius if wrong (real hardware corruption risk, unlike QEMU where a bad map might just work by luck).
4. **Framebuffer audit** — cheap to verify, likely already fine.
5. **Storage** — decide IDE-only-as-documented-limitation vs. AHCI driver.
6. **USB (xHCI)** — the largest single piece of new work in this item; likely deserves its own multi-session effort.
7. **SMP** — explicitly out of scope unless later revisited as its own item.

Open questions for whoever picks this up

- Is there a specific target machine in mind (a particular laptop/desktop with known chipset), or is the goal "reasonably broad real-hardware compatibility" in the abstract? A known target machine would let driver work be scoped much more concretely (e.g. "this machine's USB controller is xHCI at PCI address X").
- Is a spare USB stick and a real machine (even an old one) available yet for actual boot testing, or does this stay audit-only until then?